

Functions — Types, Inverse and Composition

A 1.5-2 hour lecture covering mapping concepts, injective/surjective/bijective functions, inverse, composition and Python.

- Recommended teaching duration: approximately 1.5-2 hours including board work, worked examples, Python demonstrations and student discussion.
- Teaching approach: concept → notation → worked example → CS interpretation → Python verification → student practice.

Lecture Roadmap

- Concept introduction and terminology
- Detailed definitions and notation
- Worked mathematical examples
- CS case studies with complete solutions
- Python implementation and verification
- Extended question bank
- 2-3 unsolved student practice questions at the end

1. Definition

A function $f:A \rightarrow B$ assigns every element of A exactly one element of B . A is the domain, B the codomain, and the actual outputs form the range.

2. Function vs Relation

Every function is a relation, but not every relation is a function. A function cannot assign one input to two different outputs.

3. One-to-One / Injective

Distinct inputs have distinct outputs. No two different domain elements map to the same output.

4. Onto / Surjective

Every element of the codomain is hit by at least one domain element.

5. Bijective

A function is bijective when it is both injective and surjective. A bijection gives a one-to-one correspondence between domain and codomain.

6. Inverse Function

A bijective function has an inverse that reverses the mapping. Inverse functions are important for reversible transformations and encoding/decoding systems.

7. Composition

For $f:A \rightarrow B$ and $g:B \rightarrow C$, $(g \circ f)(x) = g(f(x))$. Order matters.

8. Worked Algebraic Example

Let $f(x) = 2x + 1$ and $g(x) = x^2$. Then $(g \circ f)(x) = (2x + 1)^2$, while $(f \circ g)(x) = 2x^2 + 1$.

9. CS Case Study — Data Pipeline

A pipeline first normalizes text and then generates a key. Let $f(\text{strip/lowercase})$ and $g(\text{prefix with ID_})$.

Solution: output = $g(f(x))$. For 'Salam ', f gives 'salam' and g gives 'ID_salam'. Reversing the order would not produce the same result.

10. CS Case Study — Reversible IDs

A company maps employee usernames to unique numeric IDs and wants to recover the username from the ID.

Solution: the mapping must be injective for a unique reverse lookup; if the codomain consists exactly of assigned IDs, the mapping is bijective between those finite sets.

11. Python Functions

Python functions are computational mappings from arguments to return values. Dictionaries can implement finite mappings and inverse dictionaries when values are unique.

Python Laboratory

Python Activity 1

```
def normalize(x):
    return x.strip().lower()

def make_key(x):
    return "ID_" + x

print(make_key(normalize(" Abdul Salam ")))
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 2

```
forward={'ali':101, 'sara':102, 'hamza':103}
reverse={v:k for k,v in forward.items()}
print(reverse)
print(reverse[102])
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 3

```
def compose(g,f):
    return lambda x: g(f(x))

f=lambda x:2*x+1
```

```
g=lambda x:x*x
h=compose(g,f)
print(h(3)) # 49
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Extended Question Bank — Instructor-Led Practice

The following questions are designed to provide enough board/classroom practice for a 1.5–2 hour session. They cover the same topic areas as the relevant VU MTH202 sequence, but are written as fresh exercises rather than reproducing the handout wording.

1. Determine whether a mapping is a function.
2. Identify domain, codomain and range.
3. Classify a finite function as injective, surjective or bijective.
4. Find the inverse of a bijective mapping.
5. Find $f \circ g$ and $g \circ f$ for two algebraic functions.
6. Explain why composition order matters.
7. Model a database transformation as a function.
8. Model username-to-ID mapping and its inverse.
9. Explain why duplicate IDs prevent a unique inverse lookup.
10. Write Python functions for two transformations.
11. Create an inverse dictionary after checking value uniqueness.
12. Give a real-world example of a non-injective function.

End-of-Lecture Student Practice — No Solutions

1. Classify a finite mapping as injective/surjective/bijective and justify.
2. Find and verify an inverse.
3. Implement two composed Python transformations.

Quick Recap

Functions model mappings and computational transformations.